oldver\model3.py

```python
import torch
import torch.nn as nn
from config3 import NUM_CLASSES, USE_SE, SE_REDUCTION, USE_AUX, DROPOUT


# ── Squeeze-and-Excitation attention mechanism ──────────────────
class SEBlock(nn.Module):
    """SE: squeeze (global avg pool -> C-vector) -> excitation (FC->ReLU->FC->
    sigmoid) -> channel-wise scaling of the input tensor. Implemented from the
    lecture description, not torchvision."""
    def __init__(self, channels: int, reduction: int = SE_REDUCTION):
        super().__init__()
        hidden = max(channels // reduction, 4)
        self.squeeze = nn.AdaptiveAvgPool2d(1)
        self.excite = nn.Sequential(
            nn.Linear(channels, hidden, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(hidden, channels, bias=False),
            nn.Sigmoid(),
        )
    def forward(self, x):
        b, c, _, _ = x.shape
        z = self.squeeze(x).view(b, c)        # squeeze  -> (B, C)
        s = self.excite(z).view(b, c, 1, 1)   # excitation -> per-channel weights
        return x * s                          # channel-wise rescaling


class ConvBN(nn.Module):
    """Conv -> BatchNorm -> ReLU the BN-Inception building block"""
    def __init__(self, in_c, out_c, **kw):
        super().__init__()
        self.conv = nn.Conv2d(in_c, out_c, bias=False, **kw)
        self.bn   = nn.BatchNorm2d(out_c)
        self.act  = nn.ReLU(inplace=True)
    def forward(self, x):
        return self.act(self.bn(self.conv(x)))


# ── Inception module vs 4 parallel branches, concatenated ──────────────────
class Inception(nn.Module):
    def __init__(self, in_c, c1, c3r, c3, c5r, c5, pp, use_se=True):
        super().__init__()
        self.b1 = ConvBN(in_c, c1, kernel_size=1)                       # 1x1
        self.b2 = nn.Sequential(ConvBN(in_c, c3r, kernel_size=1),       # 1x1 -> 3x3
                                ConvBN(c3r, c3, kernel_size=3, padding=1))
        self.b3 = nn.Sequential(ConvBN(in_c, c5r, kernel_size=1),       # 1x1 -> 5x5
                                ConvBN(c5r, c5, kernel_size=5, padding=2))
        self.b4 = nn.Sequential(nn.MaxPool2d(3, stride=1, padding=1),   # pool -> 1x1
                                ConvBN(in_c, pp, kernel_size=1))
        out_c = c1 + c3 + c5 + pp
        self.se = SEBlock(out_c) if use_se else nn.Identity()
    def forward(self, x):
        x = torch.cat([self.b1(x), self.b2(x), self.b3(x), self.b4(x)], dim=1)
        return self.se(x)                     # attention applied after concat


# ── Auxiliary classifier ──────────────────────────
class AuxClassifier(nn.Module):
    def __init__(self, in_c, num_classes):
        super().__init__()
        self.pool = nn.AdaptiveAvgPool2d(4)
        self.conv = ConvBN(in_c, 128, kernel_size=1)
        self.fc1  = nn.Linear(128 * 4 * 4, 1024)
        self.drop = nn.Dropout(0.7)
        self.fc2  = nn.Linear(1024, num_classes)
    def forward(self, x):
        x = self.conv(self.pool(x))
        x = torch.flatten(x, 1)
        x = self.drop(torch.relu(self.fc1(x)))
        return self.fc2(x)
```

```python


class SamGoogLeNetEx3(nn.Module):
    """GoogLeNet-22 (BN-Inception) + optional SE attention.  Forward returns the
    main logits at eval; (main, aux1, aux2) during training when use_aux=True."""
    def __init__(self, num_classes=NUM_CLASSES, use_se=USE_SE, use_aux=USE_AUX):
        super().__init__()
        self.use_aux = use_aux

        # Stem  (224 -> 112 -> 56 -> 28)
        self.stem = nn.Sequential(
            ConvBN(3, 64, kernel_size=7, stride=2, padding=3),     # 112
            nn.MaxPool2d(3, stride=2, padding=1),                  # 56
            ConvBN(64, 64, kernel_size=1),
            ConvBN(64, 192, kernel_size=3, padding=1),
            nn.MaxPool2d(3, stride=2, padding=1),                  # 28
        )
        # Inception stacks (channel config = original GoogLeNet)
        self.inc3a = Inception(192, 64,  96,128, 16, 32, 32, use_se)   # ->256
        self.inc3b = Inception(256, 128,128,192, 32, 96, 64, use_se)   # ->480
        self.pool3 = nn.MaxPool2d(3, stride=2, padding=1)              # 14
        self.inc4a = Inception(480, 192, 96,208, 16, 48, 64, use_se)   # ->512
        self.inc4b = Inception(512, 160,112,224, 24, 64, 64, use_se)   # ->512
        self.inc4c = Inception(512, 128,128,256, 24, 64, 64, use_se)   # ->512
        self.inc4d = Inception(512, 112,144,288, 32, 64, 64, use_se)   # ->528
        self.inc4e = Inception(528, 256,160,320, 32,128,128, use_se)   # ->832
        self.pool4 = nn.MaxPool2d(3, stride=2, padding=1)              # 7
        self.inc5a = Inception(832, 256,160,320, 32,128,128, use_se)   # ->832
        self.inc5b = Inception(832, 384,192,384, 48,128,128, use_se)   # ->1024

        self.gap  = nn.AdaptiveAvgPool2d(1)        # global average pooling
        self.drop = nn.Dropout(DROPOUT)
        self.fc   = nn.Linear(1024, num_classes)

        if use_aux:
            self.aux1 = AuxClassifier(512, num_classes)   # from inc4a
            self.aux2 = AuxClassifier(528, num_classes)   # from inc4d
        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode="fan_out", nonlinearity="relu")
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1); nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                if m.bias is not None:
                    nn.init.constant_(m.bias, 0)

    def forward(self, x):
        x = self.stem(x)
        x = self.inc3b(self.inc3a(x)); x = self.pool3(x)
        x = self.inc4a(x); a1 = self.aux1(x) if (self.use_aux and self.training) else None
        x = self.inc4b(x); x = self.inc4c(x)
        x = self.inc4d(x); a2 = self.aux2(x) if (self.use_aux and self.training) else None
        x = self.inc4e(x); x = self.pool4(x)
        x = self.inc5b(self.inc5a(x))
        x = self.gap(x); x = torch.flatten(x, 1)
        x = self.fc(self.drop(x))
        if self.use_aux and self.training:
            return x, a1, a2
        return x

    def count_parameters(self):
        total = sum(p.numel() for p in self.parameters())
        train = sum(p.numel() for p in self.parameters() if p.requires_grad)
        return total, train


def build_model(use_se=USE_SE, use_aux=USE_AUX, num_classes=NUM_CLASSES):
    return SamGoogLeNetEx3(num_classes=num_classes, use_se=use_se, use_aux=use_aux)
```